



Classification with Logistic Regression

The Sigmoid Model, Decision Boundaries, and a Cost Function that Works

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 3 · Part 1

Weeks 1–2 gave us

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b \in \mathbb{R}$$

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)})^2$$

with scaling, an α search and engineered features to make it run.

Every one of those predictions was a *quantity*. Today the answer is a *category*: benign or malignant, spam or not, fraud or legitimate.

Today

1. Why linear regression breaks on classes
2. The sigmoid and the logistic model
3. Decision boundaries, linear and not
4. Why squared error fails here
5. Logistic loss and the simplified cost
6. Gradient descent, worked by hand

The recipe is unchanged — model, cost, gradient descent. All three pieces get replaced.

Motivations

Question	Input x	$y = 0$	$y = 1$
Is this e-mail spam?	word counts, sender	not spam	spam
Is this transaction fraud?	amount, time, location	genuine	fraud
Is this tumour malignant?	size, age, cell shape	benign	malignant
Will this student pass?	attendance, marks	no	yes

Binary classification

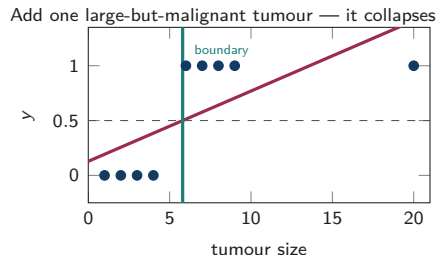
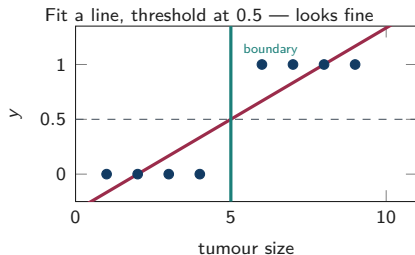
y takes exactly one of two values, written 0 and 1 by convention.

- $y = 0$: the **negative class** — absence of the thing.
- $y = 1$: the **positive class** — presence of it.

“Positive” is not “good”

A malignant tumour is the positive class. The labels name what you are *detecting*, not what you want. Choosing them the other way round is legal but inverts every coefficient's sign.

Why linear regression breaks



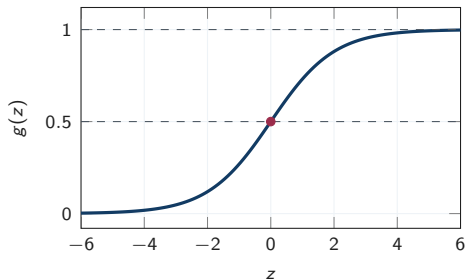
The new point is not an error — it is a perfectly ordinary large malignant tumour. Yet the least-squares line tilts to reach it, the 0.5 crossing slides from 5.0 to 5.8, and the tumour at size 6 — correctly called malignant before — is now called benign.

Key idea

Squared error punishes a confident *correct* prediction such as $f = 2.4$ for $y = 1$. Linear regression is also unbounded: $f(\mathbf{x})$ can return -3 or $+7$, which cannot be read as a class or as a probability.

The Logistic Regression Model

The sigmoid function



$$g(z) = \frac{1}{1 + e^{-z}}, \quad 0 < g(z) < 1$$

- $g(0) = 0.5$ exactly — the curve's centre.
- $g(z) \rightarrow 1$ as $z \rightarrow \infty$, $g(z) \rightarrow 0$ as $z \rightarrow -\infty$; it never reaches either.
- $g(-z) = 1 - g(z)$, so it is symmetric about $(0, 0.5)$.

z	-3	-1	0	1	3
$g(z)$	0.0474	0.2689	0.5000	0.7311	0.9526

Also called the *logistic* function — hence the name of the model.

The logistic regression model

Take the linear model of Week 2 and pass its output through the sigmoid:

$$z = \mathbf{w} \cdot \mathbf{x} + b, \quad f_{\mathbf{w},b}(\mathbf{x}) = g(z) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} \in (0, 1).$$

What the output means

$f_{\mathbf{w},b}(\mathbf{x})$ is the model's estimate of

$$P(y = 1 \mid \mathbf{x}; \mathbf{w}, b),$$

the probability that $y = 1$ for this input, given these parameters. Since y is 0 or 1,

$$P(y = 0 \mid \mathbf{x}) = 1 - f_{\mathbf{w},b}(\mathbf{x}).$$

Reading a prediction. A patient with $f_{\mathbf{w},b}(\mathbf{x}) = 0.7$: the model says there is a 70% chance the tumour is malignant, and therefore a 30% chance it is benign.

Key idea

The model outputs a *probability*, not a class. Turning it into a class needs one more decision — the threshold — and that decision is yours, not the algorithm's.

Despite the name, logistic regression is a **classification** algorithm. The “regression” is historical: it regresses the log-odds $\log \frac{p}{1-p} = \mathbf{w} \cdot \mathbf{x} + b$, which is linear in the parameters.

Decision Boundaries

Predict $\hat{y} = 1$ when $f_{\mathbf{w},b}(\mathbf{x}) \geq 0.5$. Because $g(z) \geq 0.5$ exactly when $z \geq 0$, this is the same as

$$\hat{y} = 1 \iff \mathbf{w} \cdot \mathbf{x} + b \geq 0, \quad \hat{y} = 0 \iff \mathbf{w} \cdot \mathbf{x} + b < 0.$$

Decision boundary

The set of inputs where $\mathbf{w} \cdot \mathbf{x} + b = 0$: the surface on which the model is exactly undecided. For a linear z it is a straight line in 2D, a plane in 3D, a hyperplane in general.

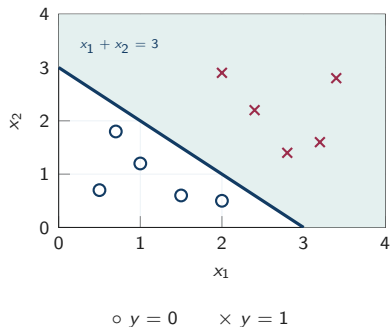
Key idea

The boundary is a property of the **parameters**, not of the training data. Two data sets that yield the same $(\mathbf{w}, b)$ have the same boundary; the data only determined which $(\mathbf{w}, b)$ was learned.

Common pitfall

0.5 is a *default*, not a law. Screening for cancer you might predict malignant at $f \geq 0.2$, accepting false alarms to miss fewer tumours; approving a loan you might demand $f \geq 0.9$. Changing the threshold moves the boundary without retraining anything.

A linear decision boundary, worked out

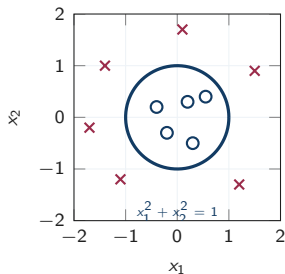


Take $\mathbf{w} = [1, 1]$ and $b = -3$, so $z = x_1 + x_2 - 3$.

boundary: $x_1 + x_2 - 3 = 0 \iff x_1 + x_2 = 3$

$\mathbf{x}$	z	$f_{\mathbf{w},b}(\mathbf{x})$	prediction
(1, 1)	-1	0.2689	$\hat{y} = 0$
(2, 1)	0	0.5000	on the boundary
(2, 2)	+1	0.7311	$\hat{y} = 1$
(3, 3)	+3	0.9526	$\hat{y} = 1$, confidently

Distance from the boundary is confidence: $|z|$ large means f near 0 or 1. The magnitude of $\mathbf{w}$ controls how fast confidence grows as you step away.



Nothing forces z to be linear in the *raw* inputs. Engineer polynomial features, exactly as in Week 2, and feed those to the same model:

$$z = w_1 x_1^2 + w_2 x_2^2 + b, \quad \mathbf{w} = [1, 1], \quad b = -1$$

$$\Rightarrow \text{boundary } x_1^2 + x_2^2 = 1, \text{ a circle.}$$

Inside the circle $z < 0$ so $\hat{y} = 0$; outside, $\hat{y} = 1$.

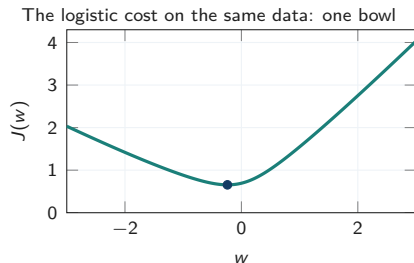
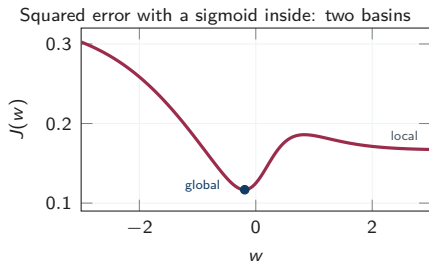
Key idea

Higher-order terms — $x_1^2 x_2$, x_1^3 , $x_1 x_2^2$ — give boundaries of almost any shape. The model stays logistic regression; only the feature vector changed.

With no engineered features the boundary is *always* a straight hyperplane, however much data you have.

The Cost Function

Squared error will not do



Both curves are the real cost for the three-point data set $x = (-1, 1, 4)$, $y = (0, 1, 0)$, with $b = 0$. On the left, gradient descent started anywhere right of $w \approx 0.8$ walks away from the better solution and settles at $J = 0.1667$ instead of 0.1168.

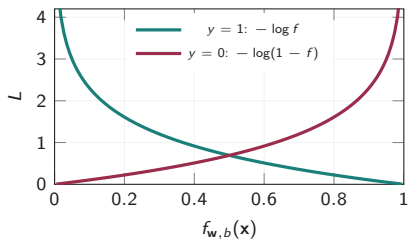
Key idea

Squared error is convex in $\mathbf{w}$ for a *linear* model. Wrapping a sigmoid around it destroys that: J acquires local minima and long flat regions, and gradient descent has no way to escape them. We need a different cost.

The logistic loss

Write the cost as an average of a per-example **loss**, so we can design the loss on its own:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\mathbf{w}, b}(x^{(i)}), y^{(i)}), \quad L(f, y) = \begin{cases} -\log(f) & \text{if } y = 1 \\ -\log(1 - f) & \text{if } y = 0 \end{cases}$$



f	L if $y = 1$	L if $y = 0$
0.99	0.0101	4.6052
0.90	0.1054	2.3026
0.50	0.6931	0.6931
0.10	2.3026	0.1054

A confidently wrong prediction costs without bound; a confidently right one costs almost nothing. That unbounded penalty is what pushes f towards the correct end, and what makes J convex.

The simplified cost function

The two branches can be written as one expression, because y is always 0 or 1:

$$L(f, y) = -y \log(f) - (1 - y) \log(1 - f)$$

Averaging over the training set gives the cost we will minimise, writing $f^{(i)} = f_{\mathbf{w}, b}(x^{(i)})$:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f^{(i)}) + (1 - y^{(i)}) \log(1 - f^{(i)}) \right]$$

Check both cases.

- $y = 1$: the second term has factor $(1 - 1) = 0$, leaving $-\log(f)$.
- $y = 0$: the first term has factor 0, leaving $-\log(1 - f)$.

One of the two terms is always switched off, so nothing has changed except that the formula is now a single differentiable expression. Note there is no $\frac{1}{2}$: it only ever cancelled the 2 from differentiating a square.

Key idea

This is not an arbitrary choice. It is the negative log-likelihood of the data under the model, so minimising J is **maximum likelihood estimation**. It is also called *binary cross-entropy*, and it is provably convex in $(\mathbf{w}, b)$.

Gradient Descent for Logistic Regression

Gradient descent for logistic regression

Repeat until convergence, updating all $n + 1$ parameters *simultaneously*:

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}) x_j^{(i)}, \quad j = 1, \dots, n$$

$$b := b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)})$$

These are algebraically identical to Week 2 — the log and the sigmoid conspire so that the derivative comes out this clean. But

$$\text{linear: } f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b \quad \text{versus} \quad \text{logistic: } f_{\mathbf{w},b}(\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}.$$

Common pitfall

Same formula, different algorithm. Everything from Week 2 still applies: simultaneous updates, feature scaling, monitoring J against iteration, the $\times 3$ search for α — and a vectorized implementation.

Four iterations, worked by hand

Data: $x = (1, 2, 3, 4)$, $y = (0, 0, 1, 1)$; one feature, $m = 4$, $\alpha = 0.5$, starting from $w = 0$, $b = 0$.

iter	$\partial J / \partial w$	$\partial J / \partial b$	w	b	$J(w, b)$
0	—	—	0.0000	0.0000	0.6931
1	-0.5000	+0.0000	0.2500	0.0000	0.6250
2	-0.0578	+0.1487	0.2789	-0.0744	0.6124
3	-0.0525	+0.1476	0.3051	-0.1482	0.6003
4	-0.0512	+0.1452	0.3307	-0.2207	0.5885

- At $w = b = 0$ every $f = g(0) = 0.5$, so $J = -\log 0.5 = 0.6931$ — the cost of guessing.
- J falls on every iteration. That is the check from Week 2, unchanged.
- Progress is slow: after four steps the boundary $-b/w$ sits at 0.67, far from the sensible 2.5. Hundreds of iterations are normal.

Key idea

$J = 0.6931 = \log 2$ is the cost of a model that predicts 0.5 for everything. Use it as a baseline: if your trained J is not clearly below $\log 2$, the model has learned nothing.

The whole model in four functions

```
def sigmoid(z):    return 1 / (1 + np.exp(-z))

def predict(X, w, b): return sigmoid(X @ w + b)          # shape (m,)

def cost(X, y, w, b):
    f = predict(X, w, b)
    return -np.mean(y*np.log(f) + (1-y)*np.log(1-f))

def gradient(X, y, w, b):
    err = predict(X, w, b) - y
    return X.T @ err / len(y), err.mean()
```

$$\log(0) = -\infty$$

With large $|z|$, sigmoid rounds to exactly 0 or 1 and the cost becomes nan. Production code uses a numerically stable form; for the lab, clip f into $[10^{-15}, 1 - 10^{-15}]$.

Compare with Week 2: only predict and cost changed. gradient is character-for-character the linear-regression version, because the mathematics says it must be.

Wrap-up and Assessment

Concepts

- Classification predicts a category; $y \in \{0, 1\}$.
- Linear regression fails: unbounded output, and the boundary moves when a far-away correct point is added.
- $f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b)$ estimates $P(y = 1 \mid \mathbf{x})$.
- Thresholding at 0.5 makes the boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$; polynomial features bend it.
- Squared error is non-convex here; logistic loss is convex.
- The gradient formula is unchanged from Week 2 — f is not.

Formulas

$$g(z) = \frac{1}{1 + e^{-z}}$$
$$J = -\frac{1}{m} \sum_i \left[y^{(i)} \log f^{(i)} + (1 - y^{(i)}) \log(1 - f^{(i)}) \right]$$
$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_i (f^{(i)} - y^{(i)}) x_j^{(i)}$$

Next

Part 2: what goes wrong when a model fits the training data too well, and how regularization fixes it — for both regression and classification.

1. Define binary classification, and state which class is conventionally labelled 1. Give two examples from outside the slides. [4]
2. Using the tumour example, explain two distinct reasons linear regression is unsuitable for classification. [6]
3. State the sigmoid function, its range, and its value at $z = 0$. Prove that $g(-z) = 1 - g(z)$. [5]
4. Write the logistic regression model and state precisely what its output means. If $f_{\mathbf{w},b}(\mathbf{x}) = 0.3$, what does the model believe about this example? [5]
5. Show that thresholding $f_{\mathbf{w},b}(\mathbf{x})$ at 0.5 is equivalent to thresholding $\mathbf{w} \cdot \mathbf{x} + b$ at 0. [5]
6. For $\mathbf{w} = [1, 1]$ and $b = -3$, give the equation of the decision boundary and compute $f_{\mathbf{w},b}(\mathbf{x})$ at $\mathbf{x} = (1, 1)$ and $\mathbf{x} = (2, 2)$. [6]

7. Explain how polynomial features let logistic regression produce a circular decision boundary. Give $\mathbf{w}$ and b for the boundary $x_1^2 + x_2^2 = 4$. [6]
8. Why is squared error unsuitable as the cost for logistic regression? Refer to convexity and to the behaviour of gradient descent. [6]
9. Write the logistic loss in its two-case form and in its single-line form, and show the two agree for $y = 0$ and for $y = 1$. [6]
10. Compute $L(f, y)$ for $(f, y) = (0.9, 1)$, $(0.1, 1)$ and $(0.1, 0)$. Comment on the relative sizes. [5]
11. Write the gradient descent updates for logistic regression. State what is identical to linear regression and what is not. [6]
12. An untrained logistic model has $J = \log 2 \approx 0.693$. Explain why, and how you would use this number when debugging a training run. [5]

1. **Design the threshold.** A Chattogram hospital will use your model to decide which patients get a biopsy. Missing a malignant tumour is far worse than an unnecessary biopsy. Choose a threshold, justify it, and describe how you would present the trade-off to the clinicians who must sign off on it. [12]
2. **Design a feature set for a decision boundary.** Sketch two-dimensional data for which no straight line separates the classes, then specify the engineered features that would let logistic regression separate it. State the price you pay for adding them. [12]
3. **Design a numerical-safety layer.** Your cost function returns `nan` after 300 iterations. Design a defensive implementation: what you clip, what you assert, and what you log so the cause is identifiable from the log alone. [10]
4. **Design a test suite.** Propose four tests for a logistic regression implementation that would catch, respectively: a missing sigmoid, a sign error in the gradient, non-simultaneous updates, and unscaled features. For each, state the input and the observable symptom. [12]
5. **Design for accountability.** A bank must tell a rejected applicant why. Given a trained logistic model, describe what you can honestly report from $\mathbf{w}$, b and $\mathbf{x}$, and one thing that would be dishonest to claim. [10]

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§4.1–4.3** — why not linear regression, the logistic model, the logistic function, estimating the coefficients by maximum likelihood, and multiple logistic regression.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§4.3.2** — logistic regression, the cross-entropy error function, and the derivation of the gradient that makes it look like the linear case.

Lab (CSE 816, Week 3): classification; the sigmoid function; decision boundary; logistic loss and cost.

Practice quizzes: *Classification with logistic regression*; *Cost function for logistic regression*.